

Flow Matching to Class Prototypes

A learned transport between the frozen feature and the prototype classifier

Yousef Shihade · Mira Bitar | University of Haifa · 2026

In Stage 2 we insert a flow-matching layer between the frozen image feature and the prototype classifier we built in Stage 1. A small velocity network learns to transport each feature toward the prototype of its class, and the transported feature is then classified with the same cosine rule as before. Everything downstream of the encoder is reused unchanged, so any change in accuracy is attributable to the flow-matching (FM) layer alone. The report follows the order of the work itself: a short refresher on where Stage 1 left off, what we set out to test, and then one section per pipeline step.

+22.3 pp

The FM layer works. Its best cell is DINOv2/Aircraft at full data, and 24 of the 36 FM cells improve on the prototype baseline.

4 of 18

Rolled-out training does not beat standard FM. It wins only four head-to-head cells, despite removing the train/inference mismatch it was designed to remove.

$\rho = 1.000$

One quantity predicts every cell. The gain tracks the headroom Stage 1 left behind, and that same quantity explains the two cells where FM loses.

1 Where Stage 1 left off

Stage 1 froze two pretrained encoders, cached their features once, and built two classifiers on top without ever fine-tuning the encoder. Stage 2 changes exactly one thing about that setup: it inserts a learned transport before the classification rule. The encoder, the cached features, the k-shot subsets, the seeds, the test splits and the prototypes are all reused unchanged, so every number in this report reads as a difference against the prototype column of Table 1 with nothing else varying.

Datasets	DTD (47 texture classes, partition 1) · FGVC-Aircraft (100 variants)
Encoders	ResNet-18 (512-d, both datasets) · DINOv2 ViT-S/14 (384-d, Aircraft)
Combinations	3 encoder/dataset pairs carried through every experiment
Training sizes	$K = 5, 10, \text{full}$ · 3 balanced random subsets per K , seeds 0/1/2
Test set	The complete official test split (1880 / 3333 / 3333), never subsampled
Reported as	Top-1 accuracy, mean $\pm$ sample std over 3 seeds

Stage 1 produced two baselines on this protocol. The linear probe, $s = Wz + b$, is a trained classifier on the frozen features and serves as the reference for how much the features can give when a classifier is actually fitted. The prototype classifier keeps one mean feature vector per class, trains nothing, and classifies by cosine similarity to the nearest prototype. It is Stage 2’s baseline, and its prototypes are the targets the flow transports toward.

Encoder / dataset	K	Linear probe	Prototype (baseline)	Headroom
ResNet-18 / DTD	5	44.86 ± 1.83	46.10 ± 1.89	-1.24
	10	52.04 ± 1.16	51.67 ± 1.33	$+0.37$
	full	62.82 ± 0.42	58.83 (1 run)	$+3.99$
ResNet-18 / Aircraft	5	20.76 ± 0.26	16.46 ± 0.39	$+4.30$
	10	27.87 ± 1.33	20.19 ± 1.14	$+7.68$
	full	38.15 ± 0.29	25.47 (1 run)	$+12.68$
DINOv2 / Aircraft	5	38.19 ± 2.49	23.79 ± 0.47	$+14.40$
	10	51.54 ± 1.70	27.74 ± 0.98	$+23.80$
	full	67.55 ± 0.28	34.26 (1 run)	$+33.29$

Table 1. The Stage 1 numbers Stage 2 has to beat (top-1 test accuracy, %). Headroom is the gain of the trained probe over the untrained prototype on the same cell.

The headroom column deserves a moment. It is small or negative on DTD, where the class mean is already a good summary of a texture, and very large on DINOv2/Aircraft. We come back to it at the end of the report, because it turns out to predict the Stage 2 result exactly.

2 What Stage 2 set out to test

Stage 2 trains a velocity network $v_\theta(z, t)$ under two objectives. Standard flow matching supervises the velocity at random points along the straight line between a feature and its prototype; rolled-out flow matching runs the full Euler integration during training and supervises only where the feature ends up.

	Standard FM	Rolled-out FM
Sampling	$t \sim \mathcal{U}(0, 1), z_t = (1 - t)z_i + tp$	full T -step Euler rollout from z_i
Target	velocity $u_i = p - z_i$	final position p
Loss	$\ v(z_t, t) - u_i\ ^2$	$\ \hat{z}_T - p\ ^2$
Backprop through	one velocity prediction	all T predictions
Train / inference	mismatched	matched
Models needed	1 (serves every T)	2 (one per T)

Table 2. The two objectives. Inference is identical for both: T Euler steps $\hat{z}_{k+1} = \hat{z}_k + \frac{1}{T}v(\hat{z}_k, \frac{k}{T})$ for $k = 0 \dots T - 1$, then $\hat{z}_T$ is classified by cosine similarity to the same prototypes. We evaluate at $T = 4$ and $T = 12$.

#	Result	Where
1	Accuracy table, ΔAcc vs. baseline, accuracy-vs- K plot with error bars	Section 6
2	Representative training-loss curves, both objectives	Section 6
3	Feature space: original vs. standard FM vs. rolled-out, jointly fitted projection	Section 7
4	Flow trajectories: intermediate states, original, transported, prototype	Section 7
opt.	The learned flow in reverse, starting from prototypes	Section 7
opt.	Samples vs. prototypes at intermediate flow times	Section 7

Table 3. Results and where each one is produced.

We also fixed the protocol in four ways. First, the datasets, encoders, prototypes, subsets, seeds and test sets are the same as in Stage 1; Section 3 re-derives the Stage 1 baseline from scratch and reproduces all 21 runs exactly ($\max |\Delta| = 0.00\text{e}+00$), and the prototypes are byte-identical to Stage 1’s saved file. Second, the architecture and the main training choices are the same for both objectives: they go through one shared setup function, so for a given seed they start from identical weights and see identical batch order, and gradient clipping was applied to both rather than only to the one that needed it. Third, the repetition protocol is Stage 1’s, three seeds everywhere; the Stage 1 full-data baseline is a single deterministic run with nothing left to subsample, so it carries no error bar, but every other cell does. Fourth, we did no hyperparameter search: the MLP is our fixed architecture (2×512 , SiLU, t concatenated to the input), deliberately not tuned.

The resulting grid is 3 encoder/dataset combinations $\times$ 3 training sizes $\times$ 3 seeds. Standard FM needs one model per cell, since it is T -independent, while rolled-out FM needs one model per T . That is 81 trained models filling a 45-cell results table (3 combinations $\times$ 3 $K \times$ 5 methods).

3 Setup and prototypes STEP 01

The whole comparison rests on the Stage 2 baseline column being the same quantity that Stage 1 reported, so before building on Stage 1 we reproduced it. Step 01 recomputes all 21 prototype runs from the cached features rather than copying the numbers across, and every one of them lands on the $y = x$ line (Figure 1).

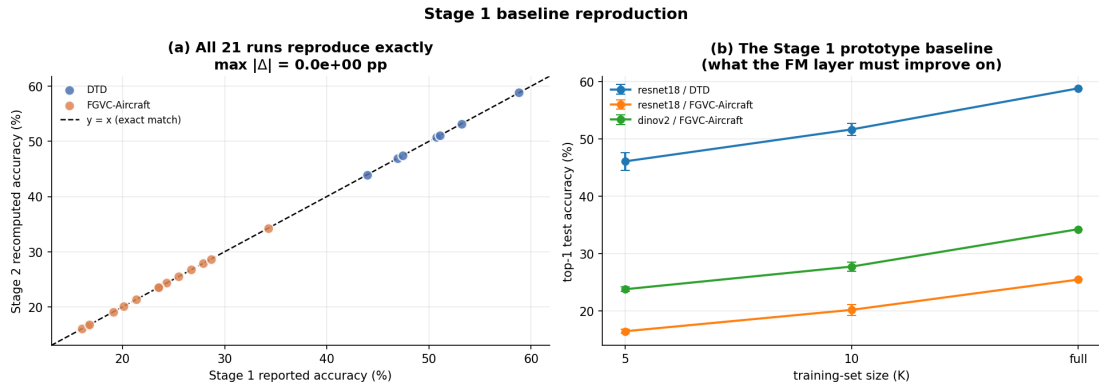


Figure 1. Stage 1 baseline reproduction. (a) All 21 runs reproduce exactly, $\max |\Delta| = 0.00\text{e}+00$. (b) The prototype baseline the FM layer has to improve on. The prototypes themselves are byte-identical to Stage 1’s saved file, so the flow is aimed at exactly the targets Stage 1 built.

The one design decision: L2-normalise before the flow

Prototypes are unit-norm by construction. Raw encoder features are not; they sit at norms of 24 to 50. The flow target is $u = p - z$, so when $\|z\| \approx 30$ and $\|p\| = 1$, the prototype is lost in the rounding and u is very nearly just $-z$. We measured this with $a = \cos(u, -z)$, the alignment between the supervised direction and “just cancel the feature”. On raw features $a = 0.9995\text{--}0.9999$, which means the target carries almost no class information. After normalising it drops to 0.22–0.35 (Figure 2).

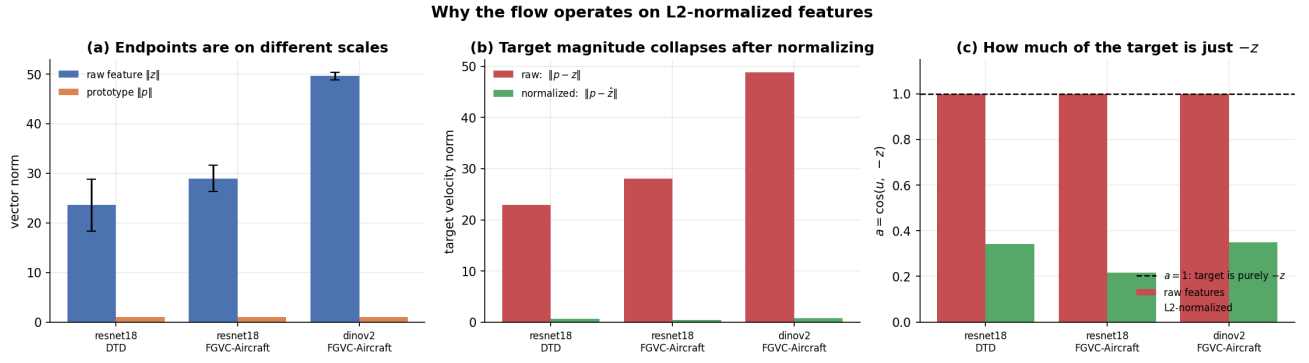


Figure 2. Why the flow operates on L2-normalised features. (a) Raw feature and prototype norms. (b) The target velocity’s magnitude collapses after normalising. (c) How much of the target is just $-z$.

MEASURED, NOT ARGUED

We did not leave this as a geometric argument. Step 02 retrains the identical objective with normalisation switched off. Raw features cost 15.8–20.3 pp, and on both ResNet-18 combinations the flow ends up worse than no flow at all (Figure 5). The prediction from the geometry and the measured outcome agree.

4 Standard flow matching STEP 02

Standard FM supervises the velocity at random points on the ideal straight path between a feature and its prototype. Training samples a random t , places z_t on the line from feature to prototype, and asks the network for that line’s constant velocity $u = p - z$. Figure 3 makes the objective visible: the network only ever sees points on those straight segments.

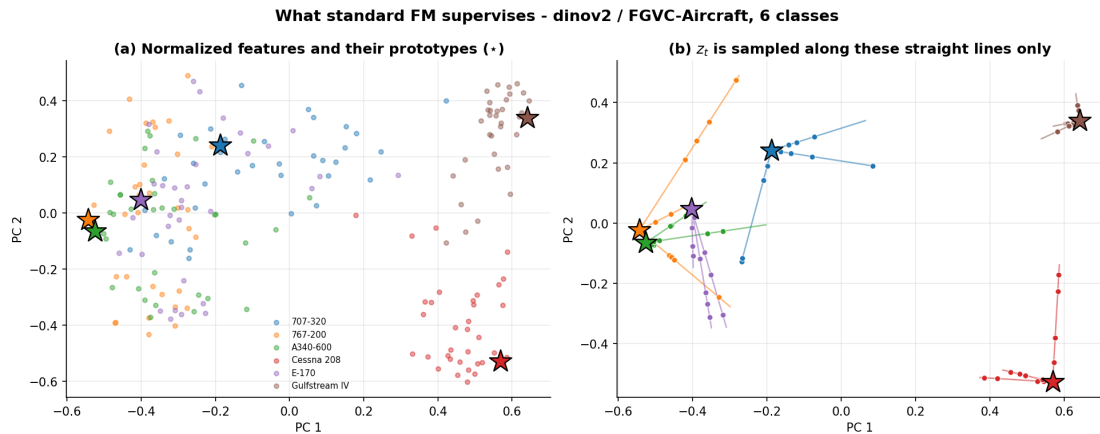


Figure 3. What standard FM supervises, DINOv2/Aircraft, six classes. (a) Normalised features and their prototypes. (b) z_t is sampled along these straight lines only.

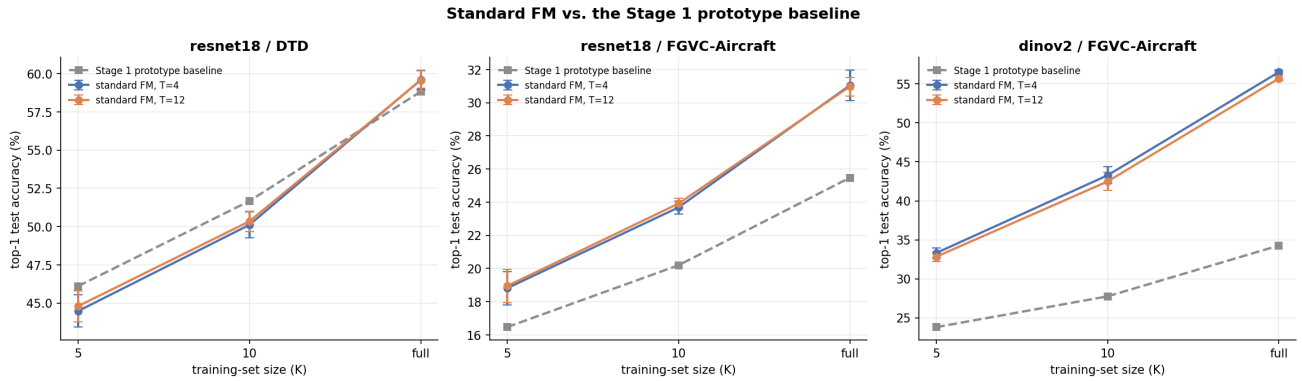


Figure 4. Standard FM at both T against the Stage 1 prototype baseline, error bars over 3 seeds. Large gains on Aircraft and essentially none on DTD, where at $K = 5$ and 10 the layer is slightly harmful.

The best cell is DINOv2/Aircraft at full data, +22.3 pp (34.26 → 56.52). The worst is ResNet-18/DTD at $K = 10$, −1.6 pp. Between the two step counts there is very little to choose: $T = 4$ and $T = 12$ differ by under 1 pp everywhere, with inconsistent sign.

We also used this step to check whether the normalisation decision from Section 3 was actually necessary, by training the same objective on raw features. It costs 15.8–20.3 pp, and the loss tells the same story: it settles at 0.08–0.20 on unit-norm inputs and at 27–111 on raw ones.

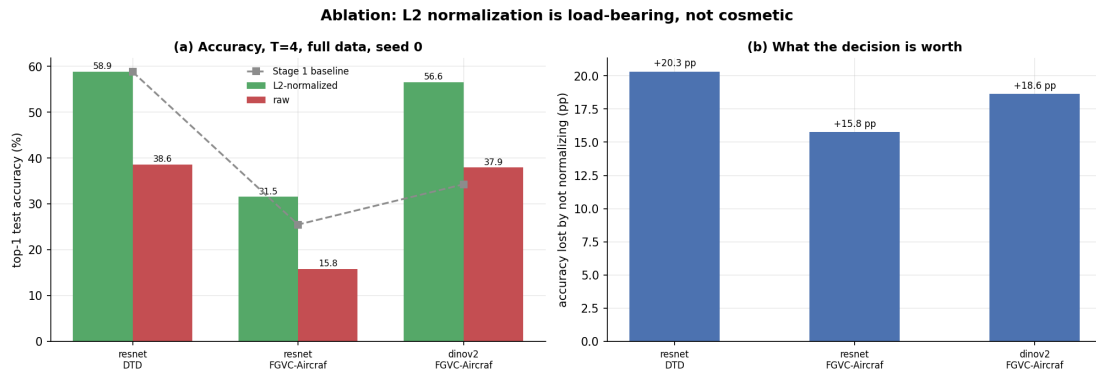


Figure 5. Ablation on L2 normalisation ($T = 4$, full data, seed 0). (a) Accuracy with and without normalisation against the Stage 1 baseline. (b) What the decision is worth in percentage points.

5 Rolled-out flow matching STEP 03

Rolled-out FM trains on exactly the states inference produces and supervises only where the feature ends up. The problem it is designed to fix is easy to see (Figure 6). Standard FM only ever sees points on the ideal straight line, but at inference the network is fed states generated by its own previous predictions, and those drift off that line. Nothing in its training taught it how to recover.

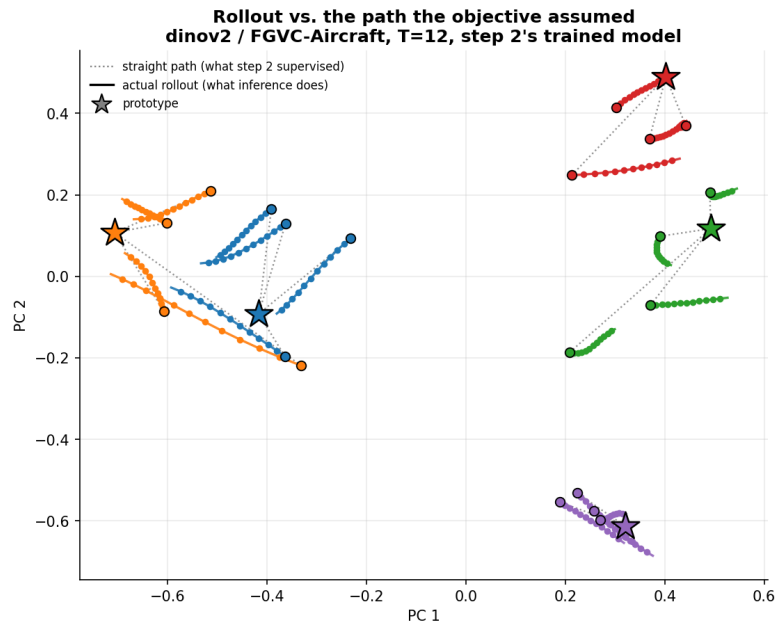


Figure 6. Rollout versus the path the objective assumed (DINOv2/Aircraft, $T = 12$, the model trained in step 02). Dotted: the straight path that was supervised. Solid: the actual rollout at inference.

Despite removing that mismatch, rolled-out FM does not help. Against standard FM at the same T it wins only 4 of 18 head-to-head cells, and on DTD it loses badly, down to -6.9 pp (Figure 7).

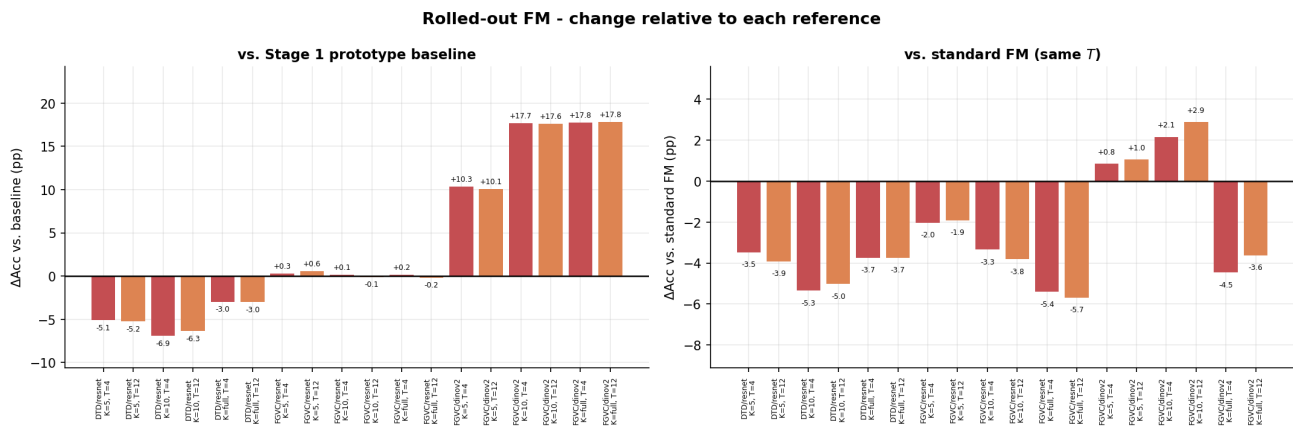


Figure 7. Rolled-out FM against the Stage 1 prototype baseline (left) and against standard FM at the same T (right).

Why not: it transports better but decides worse

The failure is not one of transport. Rolled-out FM brings features closer to their target prototype than standard FM does, on training data and on test data alike, so this is not overfitting; the layer is doing what it was trained to do (Figure 8). The catch is in what else moves. It raises the similarity to the correct prototype and to the best competing prototype by nearly the same amount (Figure 9), and only the gap between the two decides the label. A better transport is not the same thing as a better decision.

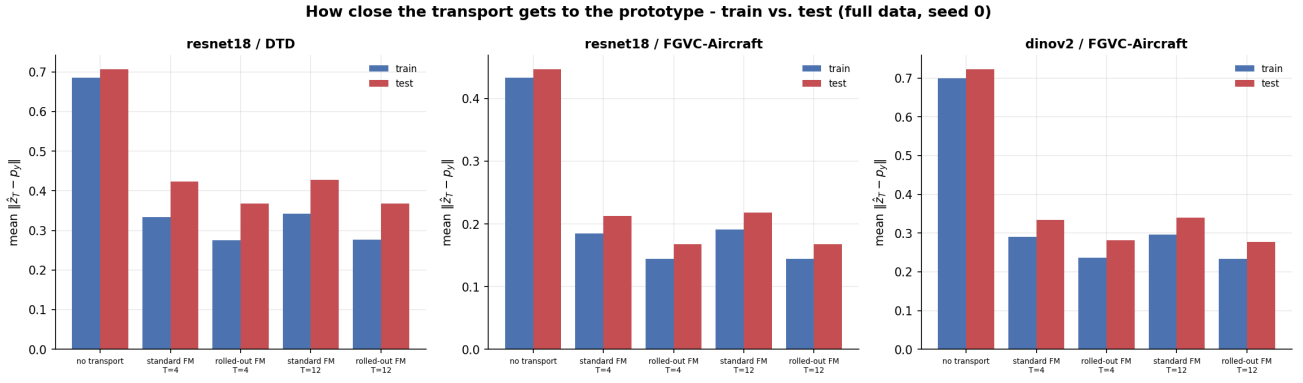


Figure 8. Distance from the transported feature to the target prototype, train and test (full data, seed 0). Rolled-out FM gets closest in every setting.

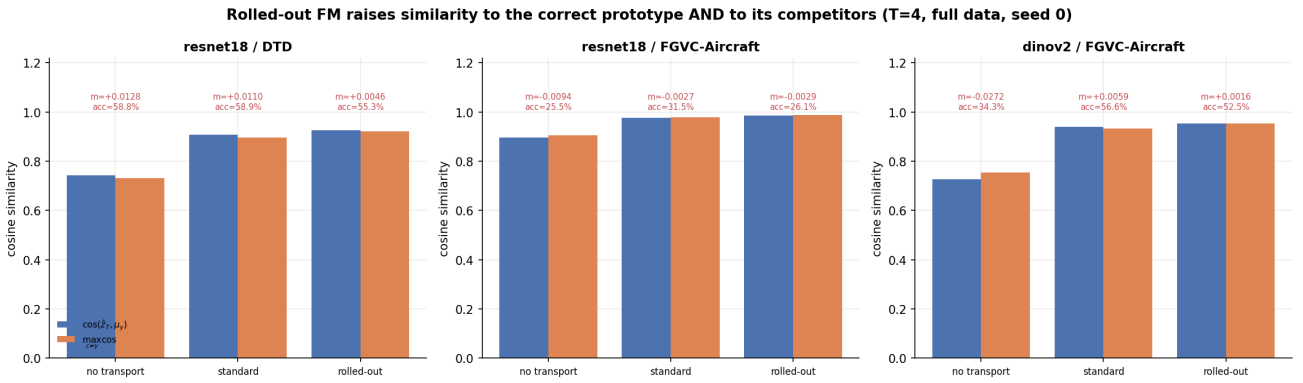


Figure 9. Cosine similarity to the correct prototype and to the best competing one ($T = 4$, full data, seed 0). Both rise together under both objectives; the margin annotated above each pair is what decides the label.

ONE RUN DIVERGED

Backpropagating through T composed Euler steps can amplify gradients. One of the 54 rolled-out runs blew up late in training, while standard FM never did in its 27. We added gradient-norm clipping at 1.0 and applied it to both objectives, not just the one that needed it, so the two stay configuration-identical. For standard FM it is inert (every cell moved by ≤ 0.25 pp). We kept the unclipped run as evidence.

6 Evaluation STEP 04

Table 4 is the complete 45-cell result. Bold marks the best FM cell in a row when it beats the baseline; three rows have no bold because no flow configuration beats it there. In total 24 of the 36 FM cells improve on the baseline, with changes ranging from +22.26 pp down to -6.90 pp.

Encoder / dataset	K	Baseline	std T=4	std T=12	roll T=4	roll T=12
ResNet-18/DTD	5	46.10 $\pm$ 1.89	44.47 $\pm$ 1.05	44.79 $\pm$ 1.01	40.99 $\pm$ 1.60	40.87 $\pm$ 1.46
	10	51.67 $\pm$ 1.33	50.11 $\pm$ 0.86	50.34 $\pm$ 0.66	44.77 $\pm$ 1.58	45.32 $\pm$ 1.23
	full	58.83 (1 run)	59.59 $\pm$ 0.61	59.56 $\pm$ 0.68	55.85 $\pm$ 0.47	55.82 $\pm$ 0.32
ResNet-18/Aircraft	5	16.46 $\pm$ 0.39	18.81 $\pm$ 1.01	18.94 $\pm$ 1.02	16.77 $\pm$ 0.84	17.03 $\pm$ 0.87
	10	20.19 $\pm$ 1.14	23.68 $\pm$ 0.39	23.91 $\pm$ 0.32	20.34 $\pm$ 0.53	20.09 $\pm$ 0.24
	full	25.47 (1 run)	31.04 $\pm$ 0.93	30.97 $\pm$ 0.56	25.64 $\pm$ 1.72	25.28 $\pm$ 1.69
DINOv2/Aircraft	5	23.79 $\pm$ 0.47	33.30 $\pm$ 0.68	32.82 $\pm$ 0.57	34.13 $\pm$ 0.59	33.86 $\pm$ 0.58
	10	27.74 $\pm$ 0.98	43.27 $\pm$ 1.09	42.48 $\pm$ 1.16	45.42 $\pm$ 1.10	45.35 $\pm$ 0.83
	full	34.26 (1 run)	56.52 $\pm$ 0.27	55.70 $\pm$ 0.31	52.05 $\pm$ 0.91	52.08 $\pm$ 0.82

Table 4. Top-1 test accuracy (%) for every combination, K and method; mean $\pm$ std over 3 seeds. Bold: best FM cell in the row, shown only where it beats the baseline.

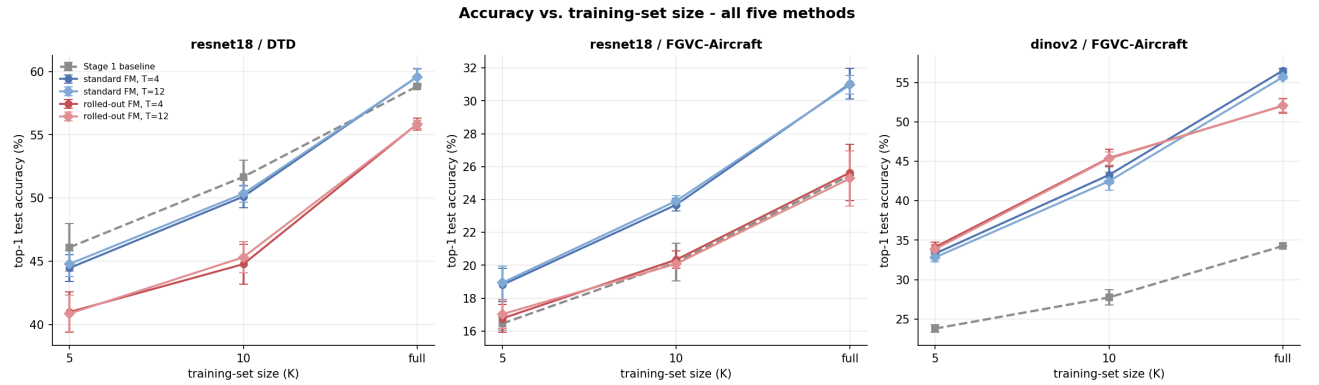


Figure 10. Result 1. All five methods against training-set size, error bars over 3 seeds. The three panels tell three different stories: FM is flat-to-harmful on DTD, clearly positive on ResNet-18/Aircraft, and transformative on DINOv2.

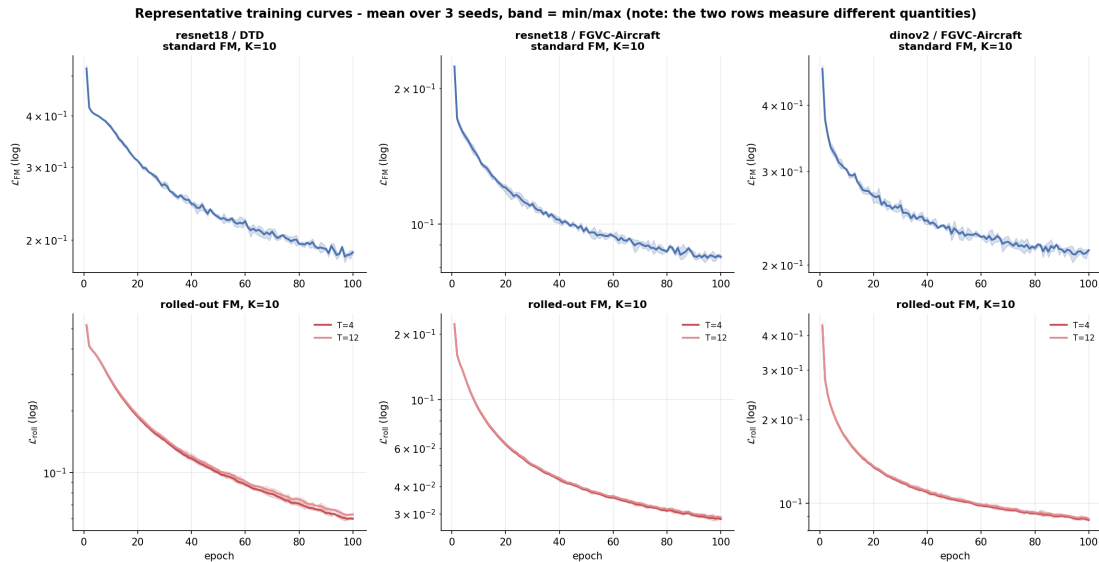


Figure 11. Result 2. Loss per epoch for both objectives, mean over 3 seeds with a min/max band. Both converge smoothly and neither is still improving at the end of the budget. The two rows sit on different scales because they measure different things, velocity error versus endpoint distance.

So when does the FM layer actually help?

The gains range from -6.9 to $+22.3$ pp. Feature separability does not order them, and K orders them only within a combination. The quantity that orders all nine cells at once is the headroom from Table 1: how much a trained linear probe gained over the untrained prototype on that same cell. The nine cells rank identically by headroom and by gain (Spearman $\rho = 1.000$), the linear fit has $R^2 = 0.988$, and it breaks even at 2.2 pp of headroom. The only two cells below that line are exactly the two where FM loses (Figure 12).

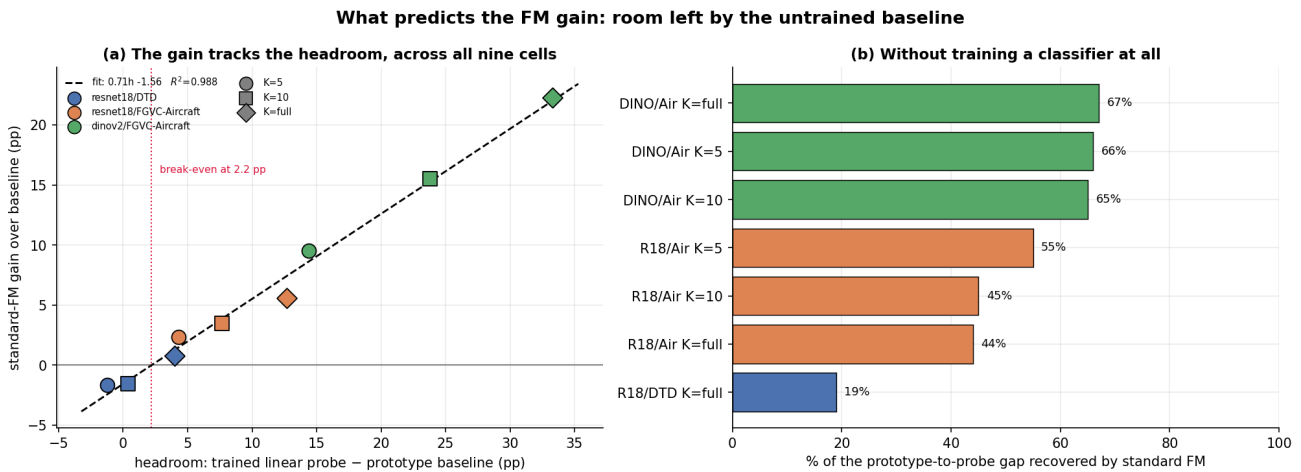


Figure 12. What predicts the FM gain. (a) Standard-FM gain over the baseline against headroom, all nine cells, with the linear fit and its break-even point. (b) The fraction of the prototype-to-probe gap that standard FM recovers, without training a classifier at all.

READ PLAINLY

The FM layer recovers roughly 70% of whatever a trained linear probe would have gained over the prototype baseline, minus a fixed cost of about 1.6 pp, while keeping the prototype structure and training no classifier at all. This also explains the DTD failures: there the prototype baseline is already level with the trained probe, so there is nothing left for a transport to recover.

Two caveats we would state out loud: nine cells is a small sample, and the linear probe is a reference point, not a mechanism.

7 Feature-space visualisations STEP 05

Results 3 and 4, plus both optional explorations.

Feature space before and after

Figures 13 and 14 show the same test images and the same class colours, with the prototypes drawn in every panel. One PCA is fitted on the concatenation of all three feature sets and the prototypes, so the three panels are directly comparable rather than three independent rotations. The success case and the failure case look surprisingly alike, and that resemblance is the point: contraction toward the right prototype is not the same thing as separation from the wrong ones, so the pictures alone cannot tell the two apart.

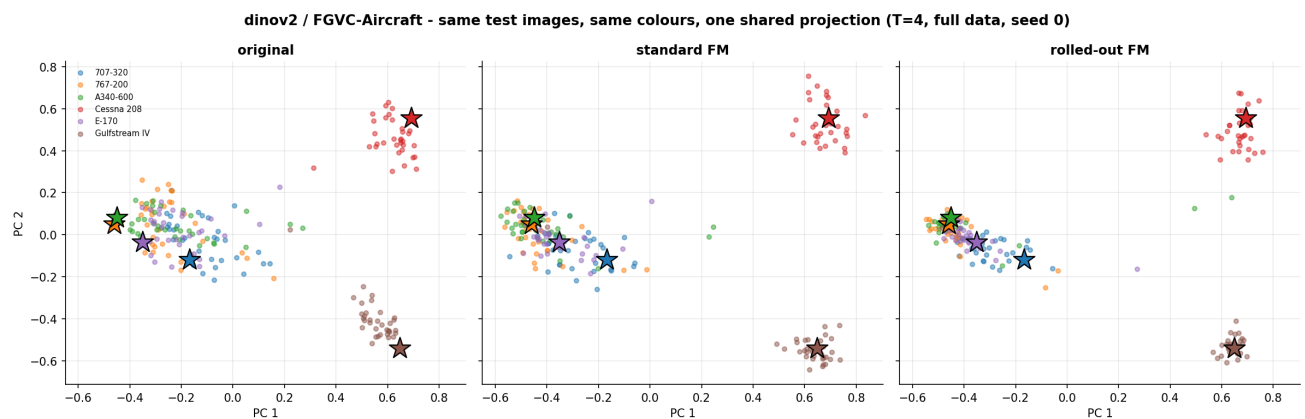


Figure 13. Result 3, the success case. DINOv2/Aircraft (+22.3 pp), $T = 4$, full data, seed 0. Both flows contract each class toward its prototype (stars), rolled-out most of all. The four overlapping classes on the left stay overlapping in all three panels.

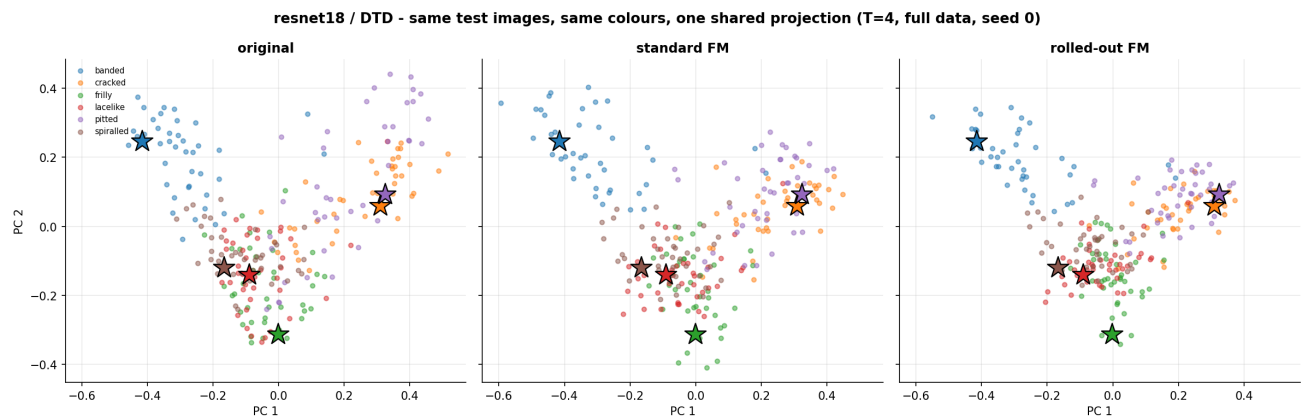


Figure 14. The failure case, ResNet-18/DTD (−1.6 pp), under the same projection scheme. It looks much the same as Figure 13.

Flow trajectories

Under a joint PCA, standard FM takes smooth, near-straight paths from each feature to its prototype. Rolled-out FM zigzags and loops (Figure 15). Only $\hat{z}_T$ is supervised, so nothing constrains the path it takes to get there, and classifying the intermediate state at every Euler step shows what that costs (Figure 16): rolled-out accuracy collapses mid-rollout, and on ResNet-18/Aircraft the exact midpoint is 9.2 pp worse than doing nothing at all, before recovering by the final step.

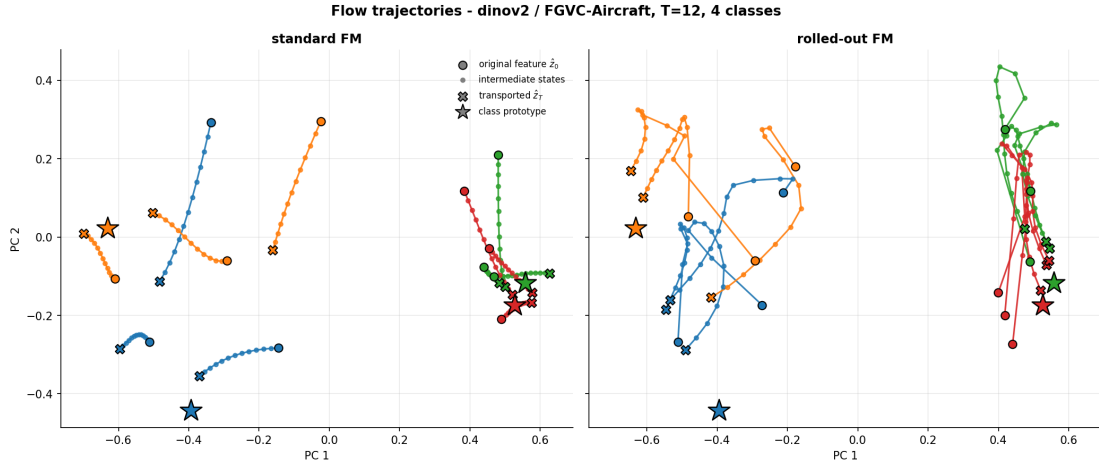


Figure 15. Result 4. Flow trajectories, DINOv2/Aircraft, $T = 12$, four classes: original feature (circle), intermediate states (small dots), transported feature (cross), class prototype (star).

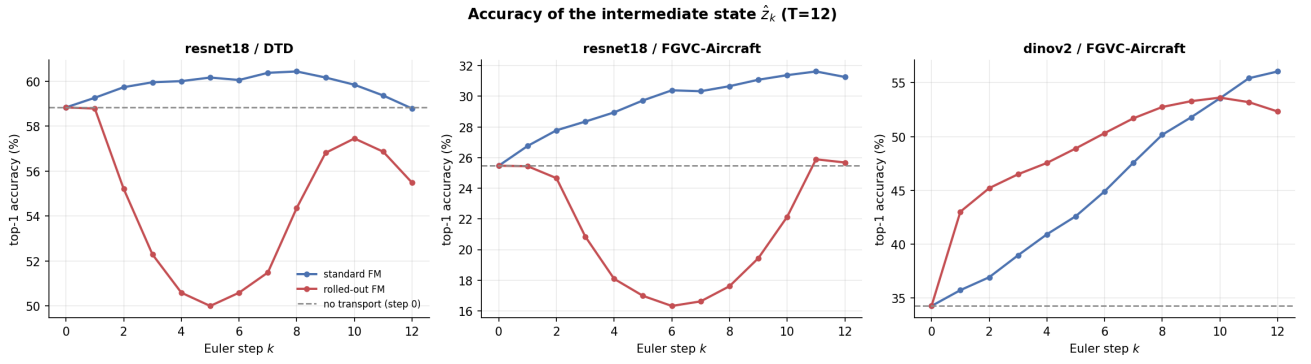


Figure 16. Top-1 accuracy of the intermediate state $\hat{z}_k$ at each Euler step ($T = 12$), against the no-transport baseline.

Tracking distances step by step (Figure 17) shows three things at once. Rolled-out FM moves features up to 149% further away from the target before returning; it nonetheless ends closer to the target than standard FM in all three settings; and the distance to the own prototype and to the nearest competing one stay glued together throughout, so only the gap between them survives, and standard FM’s gap is wider everywhere.

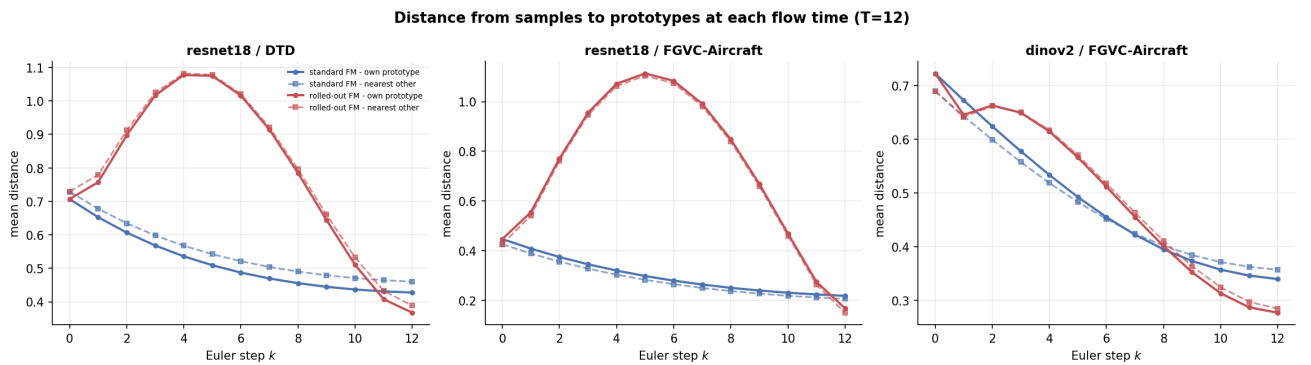


Figure 17. Optional. Mean distance from the samples to their own prototype (solid) and to the nearest competing one (dashed) at each flow time, $T = 12$.

Finally, we ran the learned field backwards, starting from the prototypes (Figure 18). The flow is a learned transport rather than an invertible generative map, and the network was never trained on reverse states, so reversing it does not recover the feature distribution it came from.

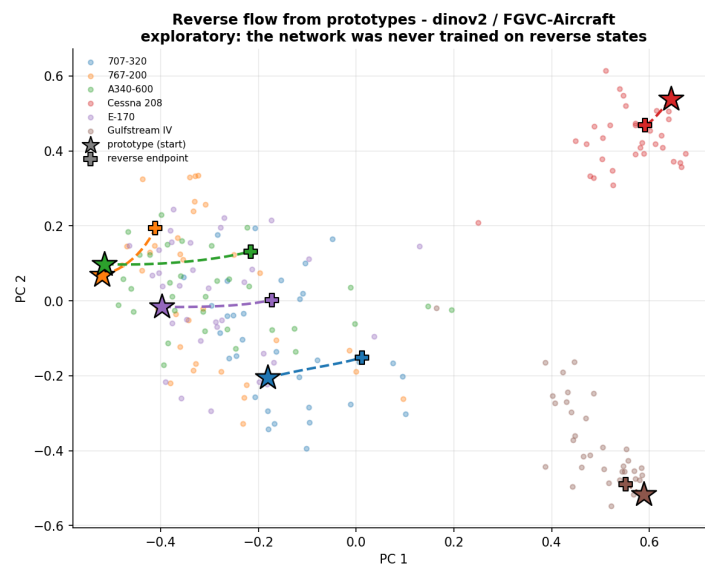


Figure 18. Optional. Reverse flow from the prototypes, DINOv2/Aircraft. Stars mark the starting prototypes and crosses the reverse endpoints.

8 What we conclude

Question	Answer
Does FM beat the prototype baseline?	Yes, where there was room: up to +22.3 pp, with 24 of 36 cells improving.
Does rolled-out beat standard FM?	No. It wins 4 of 18 cells despite removing the mismatch it targets.
How does it change the geometry?	It contracts each class toward its prototype, and toward the rival prototypes too.

Table 5. Three questions this stage set out to answer.

Five findings carry the stage. The FM layer works, and works best where the prototypes were weakest, reaching +22.3 pp. Removing the train/inference mismatch does not help: rolled-out FM reaches the prototype more closely, on test data and therefore not by overfitting, while raising similarity to competing prototypes just as much, which makes it a better transport but not a better decision. The number of Euler steps barely matters, with under 1 pp between $T = 4$ and $T = 12$ everywhere; for rolled-out FM, where T is part of training, three times the cost buys nothing. Rolled-out flows take meaningless detours, drifting up to 149% further from the target mid-rollout and losing 9.2 pp against doing nothing at the midpoint. And one quantity predicts every cell: the gain tracks the headroom Stage 1 left behind ($\rho = 1.000$, $R^2 = 0.988$), the FM layer recovers about 70% of it, and the two cells below the 2.2 pp break-even are exactly the two where FM loses.

A few choices, briefly justified

Choice	Reason
Cosine rather than Euclidean nearest-prototype	With unit-norm prototypes the two rules are identical.
No tuning of any kind	Our fixed architecture, fixed epochs, no model selection (Section 2).
No error bar on the full-data baseline	Nothing left to subsample; it is one deterministic run.
L2 normalisation before the flow	Its removal costs 15.8–20.3 pp (Figure 5), so it is doing real work.
Why rolled-out FM loses despite fixing the mismatch	The margins in Figure 9 and the distance-by-step curves in Figure 17.

What we verified before trusting any of it

- The code matches its own formulas. A test suite re-derives each of the four formulas from their stated form and checks the implementation against it, 21 tests in all, all passing. We validated the suite by breaking the code on purpose and confirming it noticed.
- Stage 1 reproduces exactly: all 21 baseline runs at $\max |\Delta| = 0.00\text{e}+00$, prototypes byte-identical.
- Every result is deterministic. Retraining all 81 models from scratch reproduced every accuracy and every loss curve bit for bit.
- Reported accuracies match the saved predictions. All 108 runs were recomputed from stored per-example predictions, with a maximum difference of $6\text{e}-06$ pp.
- The evaluation table cannot silently disagree with its own inputs. The notebook asserts that all 36 FM cells match steps 02–03 and all 9 baseline cells match Stage 1’s table, and fails rather than producing a table that drifts.